

TOOL: How Effective Are LLM-Based Approaches in Repairing Overflow Vulnerabilities

Anonymous Author(s)*

ABSTRACT

1 INTRODUCTION

Overflow vulnerabilities are a class of software weaknesses that occur when a program writes more data to a buffer than it can hold, potentially leading to serious consequences such as arbitrary code execution or memory corruption. These vulnerabilities are particularly common in low-level programming languages like C and C++, where manual memory management is widely used. Based on our analysis of two widely used vulnerability repair datasets—BigVul[3] and CVEFixes[2]—we observe that overflow-related issues account for a significant proportion of real-world vulnerabilities: 2,054 out of 4,067 CVEs (50.5%) and 7,865 out of 14,216 vulnerable functions (55.3%) are linked to overflow-related CWE entries. Furthermore, according to the 2024 CWE Top 25 Most Dangerous Software Weaknesses[10], six out of the top 25 entries fall into the category of overflow vulnerabilities, highlighting their continued prevalence and severity in modern software systems.

llm-based vulnerability repair refers to the use of large language models (LLMs) to automatically detect and fix software vulnerabilities. LLMs, trained on vast amounts of code, can understand programming languages and generate code snippets that address specific vulnerabilities. This approach leverages the capabilities of LLMs to analyze code patterns and suggest fixes, potentially improving the efficiency and effectiveness of vulnerability repair processes.

2 EMPIRICAL STUDY

2.1 Data Collection

We collect our dataset through a two-step process. First, we identify CWE entries associated with overflow vulnerabilities. Then, we extract relevant vulnerability samples from widely-used vulnerability repair datasets based on these identified CWEs.

To systematically identify overflow-related vulnerabilities, we adopt a two-pronged approach to collecting relevant CWE entries. First, we refer to the Common Weakness Enumeration (CWE) framework[12], specifically utilizing CWE View-1000[13], also known as the “Research Concepts View.” This view provides a semantically organized structure of software weaknesses tailored for academic analysis, emphasizing conceptual clarity and hierarchy. Based on this taxonomy, we identify CWE-119 — Improper Restriction of Operations within the Bounds of a Memory Buffer — and its descendant nodes as representative of overflow vulnerabilities. The complete list of selected CWE entries is provided in Table 1. Second, to complement the taxonomy-driven selection, we conduct a survey of recent research on vulnerability detection and repair. Using keywords such as “overflow,” “detect,” and “repair,” we search for papers published in top-tier software engineering and security

venues (e.g., CCS¹, ASE²). From these works[1, 4–9, 11, 14], we analyze the datasets used and extract the CWE identifiers associated with overflow vulnerabilities. We find that most of these CWEs are already included in the CWE View-1000 hierarchy; for those not covered, we incorporate them into our list to ensure comprehensive coverage and consistency with real-world research practices.

Next, we extract overflow-related vulnerability samples from two widely used vulnerability repair datasets: BigVul[3] and CVEFixes[2]. BigVul is a large-scale dataset that collects real-world vulnerabilities and corresponding fixes from open-source projects hosted on GitHub. CVEFixes, on the other hand, focuses on Common Vulnerabilities and Exposures (CVEs) and provides fine-grained fix commits associated with specific CVE entries. To maintain consistency with BigVul, we only retain C/C++ vulnerability samples from CVEFixes. We merge the two datasets following the standard practice established in prior work[15], including deduplication and partitioning at the function level. Before filtering, the merged dataset consists of 135 CWEs, 4,067 CVEs, and 14,216 vulnerable functions. By applying the set of CWE identifiers identified in the previous step, we construct a refined dataset focused on overflow vulnerabilities, which includes 2,054 distinct CVEs and 7,865 vulnerable functions. A detailed breakdown of the final dataset is presented in Table 2.

¹ACM Conference on Computer and Communications Security

²IEEE/ACM International Conference on Automated Software Engineering

ID	Name
119	Improper Restriction of Operations within the Bounds of a Memory Buffer
120	Buffer Copy without Checking Size of Input ('Classic Buffer Overflow')
121	Stack-based Buffer Overflow
122	Heap-based Buffer Overflow
123	Write-what-where Condition
124	Buffer Underwrite (Buffer Underflow)
125	Out-of-bounds Read
126	Buffer Over-read
127	Buffer Under-read
190*	Integer Overflow or Wraparound
415	Double Free
416	Use After Free
466	Return of Pointer Value Outside of Expected Range
476*	NULL Pointer Dereference
761*	Free of Pointer not at Start of Buffer
786	Access of Memory Location Before Start of Buffer
787	Out-of-bounds Write
788	Access of Memory Location After End of Buffer
805	Buffer Access with Incorrect Length Value
806	Buffer Access Using Size of Source Buffer
822	Untrusted Pointer Dereference
823	Use of Out-of-range Pointer Offset
824	Access of Uninitialized Pointer
825	Expired Pointer Dereference

Table 1: List of CWE entries related to overflow vulnerabilities. Entries marked with * are added based on recent literature.

CWE ID	CVE Count	Vulnerability Count
CWE-119	483	1532
CWE-120	71	249
CWE-121	9	14
CWE-122	47	69
CWE-125	425	1417
CWE-126	2	2
CWE-190	159	2539
CWE-415	42	162
CWE-416	215	463
CWE-476	267	456
CWE-787	319	943
CWE-823	4	5
CWE-824	11	14
Total	2054	7865

Table 2: CWE Distribution

2.2 RQ design

- **RQ0. Vulnerability Characterization.** What are the key characteristics of overflow vulnerabilities?
 - **RQ0.0 Trigger Logic.** What are the common triggering patterns or logic that lead to overflows?
 - **RQ0.1 Fixing Strategies.** What kinds of repair strategies are commonly applied to overflow vulnerabilities?

- **RQ1. Effectiveness of Simple Prompting.** How effective is a simple prompt-based approach in enabling LLMs to repair overflow vulnerabilities?
 - **RQ1.1 Impact of Trigger Logic.** How does the effectiveness of prompt-based LLM repair vary across different overflow trigger logics?
 - **RQ1.2 Impact of Fixing Strategies.** How does the effectiveness of prompt-based LLM repair vary across different fixing strategies?
- **RQ2. Effectiveness of Agent-based Repair.** How effective is an agent-assisted approach in enabling LLMs to repair overflow vulnerabilities?
 - **RQ2.1 Impact of Trigger Logic.** How does the effectiveness of agent-based LLM repair vary across different overflow trigger logics?
 - **RQ2.2 Impact of Fixing Strategies.** How does the effectiveness of agent-based LLM repair vary across different fixing strategies?

2.3 RQ0: Vulnerability Characterization

2.3.1 *exploit labeling*. xxxxx

Answer to RQ0

Overflow vulnerabilities can be characterized along two axes: the *trigger logic* and the *repair strategy*.

3 DESIGN OF TOOL

In this section, we present the design of TOOL.

4 EVALUATION

We evaluate TOOL based on the following research questions.

- **RQ1. Effectiveness Evaluation.** How is the effectiveness of TOOL compared to the state-of-the-art tools?
- **RQ2. Ablation Study.** How does each component contribute to TOOL's effectiveness?

4.1 Setup

4.2 Setup

Dataset.

Baselines.

Metrics.

4.3 Discussion

Threats. First,

Limitations. First,

5 RELATED WORK

We review previous work related to

6 CONCLUSIONS

In this paper,

REFERENCES

- [1] Jinsheng Ba, Gregory J Duck, and Abhik Roychoudhury. 2022. Efficient greybox fuzzing to detect memory errors. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 1–12.
- [2] Guru Bhandari, Amara Naseer, and Leon Moonen. 2021. CVEfixes: Automated Collection of Vulnerabilities and Their Fixes from Open-Source Software. In *Proceedings of the 17th International Conference on Predictive Models and Data Analytics in Software Engineering (PROMISE '21)*. ACM, 10. <https://doi.org/10.1145/3475960.3475985>
- [3] Jiahao Fan, Yi Li, Shaohua Wang, and Tien N. Nguyen. 2020. A C/C++ Code Vulnerability Dataset with Code Changes and CVE Summaries. In *Proceedings of the 17th International Conference on Mining Software Repositories* (Seoul, Republic of Korea) (MSR '20). Association for Computing Machinery, New York, NY, USA, 508–512. <https://doi.org/10.1145/3379597.3387501>
- [4] Floris Gorter, Taddeus Kroes, Herbert Bos, and Cristiano Giuffrida. 2024. Sticky tags: Efficient and deterministic spatial memory error mitigation using persistent memory tags. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 4239–4257.
- [5] Yiyuan Guo, Peisen Yao, and Charles Zhang. 2024. Precise Compositional Buffer Overflow Detection via Heap Disjointness. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 63–75.
- [6] Wooseok Kang, Byoungso Son, and Kihong Heo. 2022. Tracer: Signature-based static analysis for detecting recurring vulnerabilities. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. 1695–1708.
- [7] Tan Khang Le, Saba Alimadadi, and Steven Y. Ko. 2024. A Study of Vulnerability Repair in JavaScript Programs with Large Language Models. In *Companion Proceedings of the ACM Web Conference 2024* (Singapore, Singapore) (WWW '24). Association for Computing Machinery, New York, NY, USA, 666–669. <https://doi.org/10.1145/3589335.3651463>
- [8] Yuan Li, Wende Tan, Zhizheng Lv, Songtao Yang, Mathias Payer, Ying Liu, and Chao Zhang. 2022. Pacmem: Enforcing spatial and temporal memory safety via arm pointer authentication. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. 1901–1915.
- [9] Zhanpeng Liu, Yi Rong, Chenyang Li, Wende Tan, Yuan Li, Xinhui Han, Songtao Yang, and Chao Zhang. 2025. CCTAG: Configurable and Combinable Tagged Architecture. In NDSS.
- [10] MITRE. 2024. CWE Top 25 Most Dangerous Software Weaknesses for 2024. <https://cwe.mitre.org/top25/>. Accessed: 2025-07-21.
- [11] Benjamin Steenhoeck, Md Mahbubur Rahman, Richard Jiles, and Wei Le. 2023. An empirical study of deep learning models for vulnerability detection. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2237–2248.
- [12] The MITRE Corporation. 2024. *Common Weakness Enumeration (CWE)*. MITRE. <https://cwe.mitre.org/> Accessed: 2025-07-21.
- [13] The MITRE Corporation. 2024. *Research Concepts (View-1000) – Common Weakness Enumeration (CWE) Detail for CWE-1000*. MITRE. <https://cwe.mitre.org/data/definitions/1000.html> Accessed: 2025-07-21.
- [14] Junwei Zhang, Zhongxin Liu, Xing Hu, Xin Xia, and Shanping Li. 2023. Vulnerability detection by learning from syntax-based execution paths of code. *IEEE Transactions on Software Engineering* 49, 8 (2023), 4196–4212.
- [15] Xin Zhou, Kisub Kim, Bowen Xu, DongGyun Han, and David Lo. 2024. Large Language Model as Synthesizer: Fusing Diverse Inputs for Better Automatic Vulnerability Repair. *CoRR* (2024).